

Structure du Projet

Le développement de Orbi Universo 2 à été séparé en deux parties pouvant être travaillé en parallèle afin de tirer pleinement parti des compétences de toute l'équipe de développement et pouvoir rapidement identifier les aspects du jeu nécessitant une attention supplémentaire :

- Le **cœur** : sert de fondation pour toutes les mécaniques de jeu. Il est prévu pour être versatile et pouvoir supporter toutes les fonctionnalités nécessaires au Game Designers pour implémenter les règles du jeu. Il gère également le rendu graphique, menus, sons, sauvegarde et chargement de données.
- Le **contenu** : Implémenté par les Game Designers, en utilisant les capacités du langage C# et en se basant sur les fonctionnalités proposées par le cœur. Il consiste en toutes les règles du jeu et la manière dont la simulation évolue. Il est conçu pour que son implémentation soit simple et pouvoir être facilement étendu.
C'est aussi la partie du moteur de jeu qui, ultimement, sera accessible à la communauté de joueurs pour y ajouter leur propre contenu.

Le contenu est chargé en jeu à partir de fichiers au format DLL (Dynamic Link Library). Le code écrit pour le contenu réside dans un assembly à part séparé du reste du code, cela permet une compilation plus rapide lors de modifications et à la communauté de joueurs de le modifier ou y ajouter des fonctionnalités en chargeant leurs propres fichiers.

Structure du **contenu**

Les enjeux de Orbi Universo 2 on nécessité une approche unique au développement de contenu pour le jeux, qui se veux régie par son système interconnecté. Chaque mécanique nécessite une grande quantité de logique unique pour permettre au différentes facettes d'une civilisation de communiquer entre elles et de former un tout cohérent.

Nous avons mis au point une méthodologie propre aux nécessités de Orbi Universo 2 pour efficacement ajouter et tester du nouveau contenu. La création de contenu qui se veux simple à implémenter est séparé en deux parties :

- La **logique** de fonctionnement des règles du jeu, Implémenté grâce au langage de programmation C# et une méthodologie spécialement conçu pour le jeu et permet de rapidement ajouter ou modifier les règles.
- L'espace de jeu représenté par la **carte du monde**, ainsi que ses propriétés comme les différents climats et ressources naturelles, sont Implémenté en jeu grâce à un logiciel développé spécialement pour Orbi Universo 2 (détaillé par la suite dans la partie « Conceptualisation du système de carte »).
- La **localisation** contient l'ensemble des textes affichés dans la langue choisie par le joueur, sous la forme de paires clé/texte organisées dans un tableur. Chaque ligne de texte est automatiquement assignée à son *Node* correspondant grâce à sa clé représentant le nom du *Node*. Il est ainsi facile d'ajouter ou corriger n'importe quelle langue dans le jeu sans interférer avec le reste du contenu.



Simulation du monde de Orbi Universo 2

Pour représenter n'importe quel genre de civilisation, les mécaniques de Orbi Universo 2 se veulent complexes et extensives, tout en gardant le code responsable de la simulation du jeu facile à prototyper et maintenir.

La simulation est composée de plusieurs objets organisés de façon hiérarchique, chaque niveau exécutant des tâches de plus en plus détaillées.

1.  **Simulation** : contient toutes les données de la partie actuelle du joueur, plus précisément les autres objets de la hiérarchie (comme les civilisations) et des variables de portée globales (comme le temps écoulé).
2.  **Planète** : contient la topographie de la carte du monde et les données relatives au climat et ressources des différentes régions.
3.  **Civilisation** : contient les données et processus d'une seule civilisation, ainsi que tous les nœuds appartenant à cette civilisation.
4.  **Node** : le niveau le plus détaillé de la simulation. Chaque nœud représente une caractéristique propre à la civilisation à laquelle il appartient, sous la forme d'une valeur numérique qui peut fluctuer au cours du temps selon certains facteurs externes.
5.  **Flux** : permet d'échanger des informations entre différents *Nodes*. Les *Nodes* s'influencent entre eux pour faire progresser la civilisation.

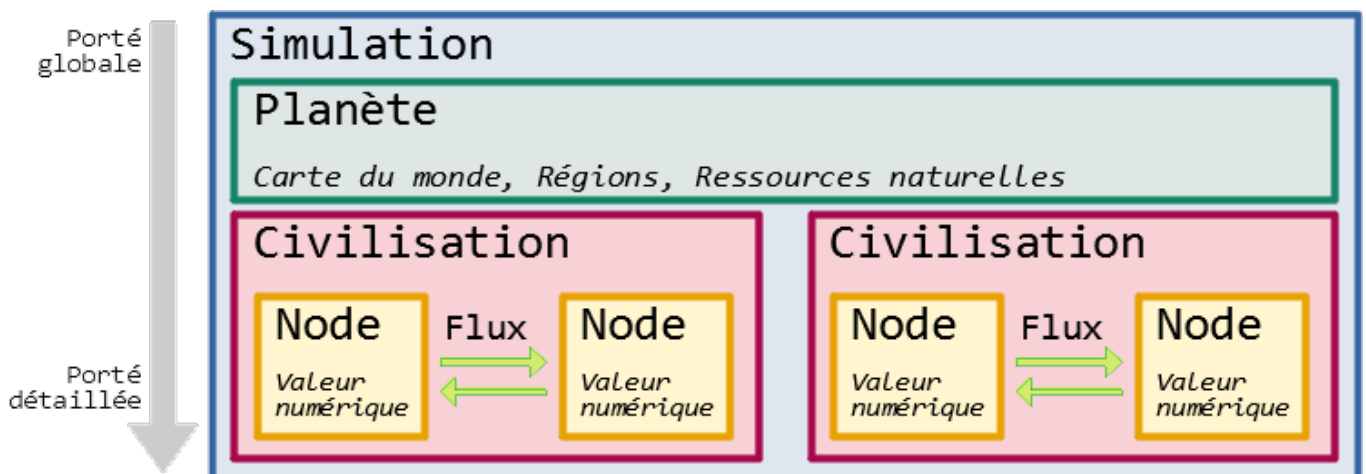


Schéma représentant un exemple de hiérarchie des objets constituant la simulation.

Cette organisation en hiérarchie à plusieurs bénéfices :

- La capacité de pouvoir ajouter ou retirer des objets en temps réel sans compromettre le déroulement de la simulation (ajouter une civilisation lors de sa création, ajouter un nœud quand une civilisation évolue ou retirer une civilisation après son effondrement).
- La capacité de sauvegarder l'état actuel de la simulation sous forme de fichier texte au format JSON (JavaScript Object Notation) adapté à une organisation hiérarchique, afin de pouvoir reprendre une partie en cours.
- La capacité de facilement ajouter des nœuds ou cartes du monde (nouveau contenu) par l'équipe de développement et, par la suite, la communauté de joueurs sans perturber le bon fonctionnement des mécaniques déjà implémentées.

Détails de l'implémentation du contenu

Le développement du contenu est principalement basé sur l'ajout de nouveaux Nœuds, car ils contiennent la logique nécessaire à la progression et au déroulement de la partie.

```
public class Exemple : TechnologyNode
```

Chaque Nœud est représenté par une classe C# qui hérite d'une classe différente en fonction de son type, lui procurant des fonctionnalités propres à chaque type de Nœud ([PolicyNode](#), [DiplomacyNode](#), [InfrastructureNode](#), [EventNode](#), [ResourceNode](#), [StateNode](#), [TechnologyNode](#), [BattleNode](#))

```
[Setup(name, icon, min, max, defaultValue)]  
[Shop(researchTime, politicalPower, wealth)]  
public class Exemple : TechnologyNode
```

La classe de chaque Nœud utilise des en-tête pour définir son état initial.

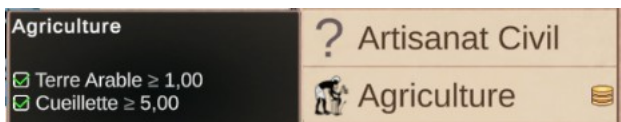
- **Setup** permet de définir le nom et l'icône affichée, ainsi que la valeur numérique par défaut au moment où le Nœud apparaît à l'écran et sa valeur maximale et minimum possible.
- **Shop** permet de définir les paramètres d'achat de ce Nœud, le coût en pouvoir politique et richesse, ainsi que le temps nécessaire pour le débloquent.

```
public bool IsHinted(Simulation, Civilization, NodeScope, Region)
```

```
public Condition[] GetUnlockConditions()
```

En plus de l'en-tête, deux méthodes permettent de déterminer si le Nœud est disponible à l'achat :

- **IsHinted** détermine si le *Node* est suggéré (visible) au joueur. Les arguments [NodeScope](#) et [Region](#) déterminent s'il s'agit d'un *Node* spécifique à une région.
- **GetUnlockConditions** fournit une liste de conditions nécessaires pour rendre le *Node* disponible au joueur, ces conditions lui sont indiquées avec un survol de la souris.



Agriculture est disponible car la liste des conditions (à gauche) est vérifiée.

```
protected void Update(float deltaTime)
```

La majorité de la logique du Nœud est comprise dans une méthode appelée 20 fois par seconde pour exécuter les processus nécessaires dans le temps. La même méthode est appelée moins souvent dans les Nœuds appartenant à des civilisations adverses contrecollées par l'ordinateur pour économiser du temps de calcul.

```
[Auto] Flux faunaFlux;
```

Les différents *Nodes* s'influencent entre eux à l'aide de *Fluxes* permettant d'augmenter ou de diminuer leur valeur. Un Flux peut être facilement créé grâce à l'en-tête **Auto**, le *Node* nommé dans la variable (ici : *faune*) sera automatiquement lié par un Flux afin de simplifier au maximum l'accès à d'autres *Nodes*.



Un Flux trace une ligne entre deux Nodes interconnectés.

Cette approche, où chaque Nœud contrôle ses propres données et sa propre logique, nous permet de rapidement développer une simulation fonctionnelle à partir d'une poignée de Nœuds et d'étendre par la suite sa complexité en ajoutant du nouveau contenu, à la fois pour notre équipe de développement et pour la communauté de joueurs souhaitant apporter leur propre contenu.

Carte et Régions

La carte du monde est un élément central et novateur de Orbi Universo 2. L'expérience de jeu est fortement impacté par le continent et le climat du lieu choisi par le joueur. C'est pourquoi nous portons une grande attention aux méthodes de conception de la carte du monde, pour que chaque nouvelle partie soit équilibrée et engageante peu importe la région du monde choisie.

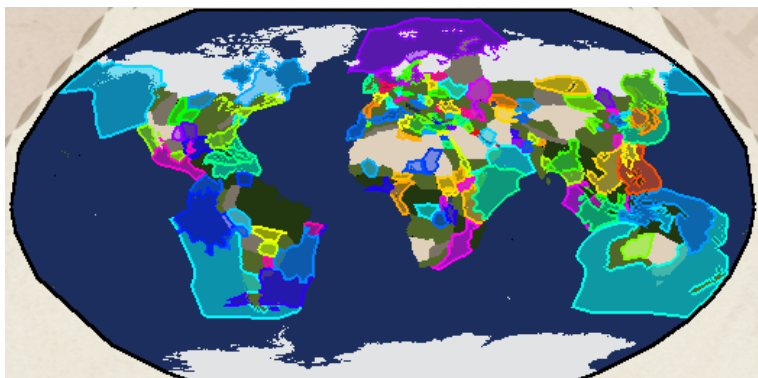
Représentation de la carte

La carte du monde est divisée en région, représentant des plaines, montagnes, océans, etc. et constitue l'unité d'espace sur la carte, Chaque région peut avoir des formes et tailles variées.

Cette division en région permet au joueur d'interagir avec la carte en cliquant sur certaines régions et d'appliquer des effets visuels en temps réel comme colorier les territoires contrôlés ou couvrir les régions inexplorées sous un brouillard de guerre.



Début de partie, ou quelques civilisations occupent peu de territoire (*affiché en rouge et vert clair*)

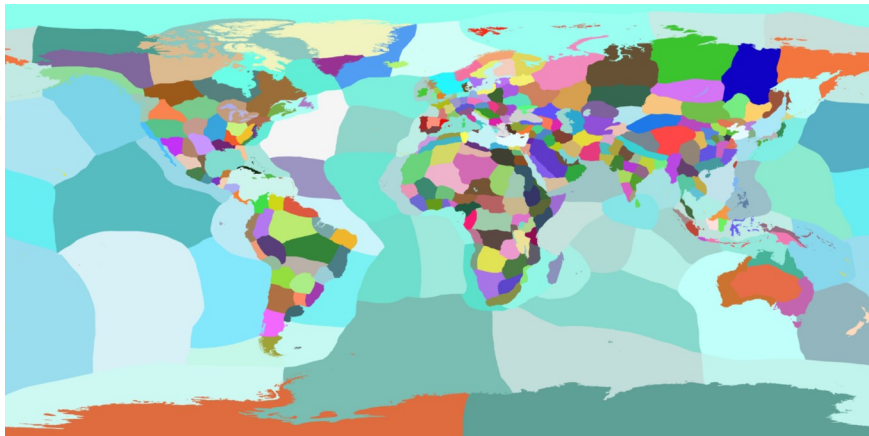


Partie avancée, ou beaucoup de civilisation se partage le globe. Le territoire de chaque civilisation est affiché d'une couleur différente.

Construction des régions

Pour les besoins de Orbi Universo 2, nous avons mis au point un processus permettant de convertir une carte du monde sous forme d'image en une carte intégrable en jeu. Cette méthode permet aussi de modifier la carte si besoin, que ce soit par l'équipe de développement ou la communauté de joueurs.

1. Une image de la carte du monde est dessinée, en utilisant une couleur différente pour chaque région.



2. L'image est convertie au format vectoriel pour séparer chaque couleur en forme distinctes.
3. Chaque forme vectorielle est convertie en maillage et sera utilisé pour afficher la carte en jeu de manière optimisée et d'apporter des modifications à celle-ci en temps réel.
4. Chaque région se voit attribué certaines propriétés comme un climat et des ressources naturelles grâce au logiciel *Gaïa* développé pour la création de Orbi Universo 2 (détaillé par la suite dans la partie « Conceptualisation du système de carte »).

La carte résultante de cette méthode peut être affichée avec n'importe quelle projection en fonction de la situation.



Projection Wagner VI



Projection Sphérique

IAs

Une innovation de Orbi Universo 2 est l'apparition d'autres civilisations en plus de celle du joueur afin de rendre le monde plus dynamique et réaliste.

Ces civilisations sont contrôlées par l'ordinateur et doivent pouvoir effectuer les mêmes actions que le joueur, avec quelques exceptions :

- La mise à jour de l'état actuel de la civilisation est ralentie pour préserver du temps de calcul, Les civilisations en contact avec le joueur sont mis à jour une fois par secondes et les autres non en contact seulement une fois toutes les 7 secondes.
- Les civilisations peuvent tricher si une règle régissant leur comportement le permet, notamment pour apparition de technologie ou de décrets, afin de compenser le manque de réactivité et de garder un avantage compétitif sur le joueur en fonction de la difficulté choisie.

Émergence des civilisations

Les autres civilisations apparaissent au cours du temps dans des régions inoccupées. Une fonction est utilisée pour déterminer quelles régions sont les plus propices à l'émergence de civilisation :

```
public float GetChanceOfEmergingCivilization(Region, CivilizationEmergenceMode)
```

Le paramètre `CivilizationEmergenceMode` est choisi par le joueur en début de partie et détermine la manière dont nouvelles civilisations apparaissent.



Personnalités

Chaque civilisation contient une liste de valeurs représentant différentes tendances comportementales à chaque instant (Impérialiste, Défensif, Diplomate, ...) Ces valeurs offrent un niveau d'abstraction supérieur dans la prise de décision de l'ordinateur.

```
public class DiplomatPersonality : PersonalityRule
```

Chaque personnalité est représentée par une classe C# qui détermine les conditions nécessaires pour que cette personnalité prenne le dessus sur les autres.

Comportement

```
public class Rule : AIRule
```

Le comportement des civilisations est régi par un ensemble de règles. Représenté par des classes C#

```
public float GetPossibilityOfExecutingRule(Personality)
```

La règle à exécuter à chaque instant est déterminée par une fonction, et varie selon les besoins de la civilisation.

```
public float[] AIAction[] GetActions()
```

Enfin, chaque règle exécute une liste d'actions basiques (*faire apparaître un Node, modifier une valeur, ...*)

Cette méthode de contrôle des civilisations adverses permet d'obtenir des comportements complexes à partir de simples règles et d'ajouter ou de modifier certaines règles sans influencer les autres déjà implémentés.

États des Lieux des Challenges techniques

Le développement de Orbi Universo 2 a présenté plusieurs challenges , Du fait des attentes techniques supérieures à l'opus précédent Orbi Unvierso 1 :

1. **Carte** : Une carte du monde segmenté en un grand nombre de régions.
2. **Civilisations** : Plusieurs civilisation évoluant en même temps et interagissant entre elles et avec l'environnement.
3. **Nodes** : Un plus grand nombre de *Nodes* interconnectées avec des interactions plus complexes.

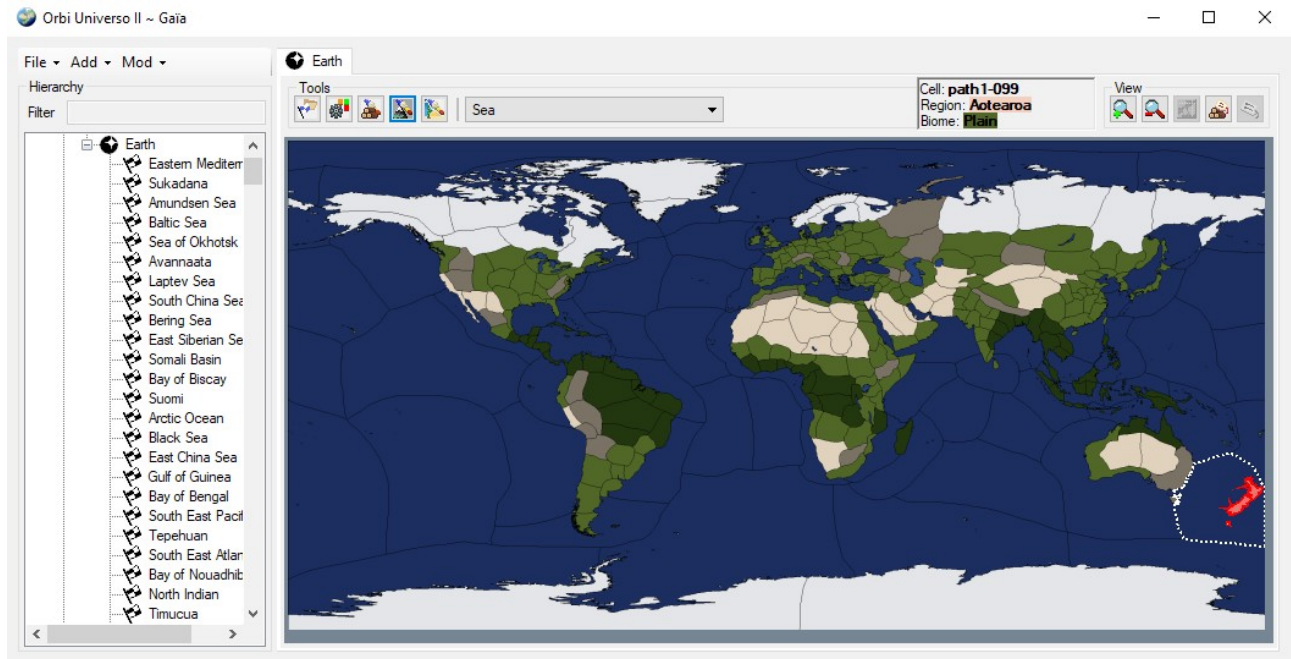
La précédente phase de prototypage de Orbi Universo 2 nous a permis de valider la faisabilité malgré ces challenges techniques :

1. **Carte** : La méthodologie précédemment exposé dans la partie « Construction des régions » permet de construire un carte du monde qui satisfait les intentions originales du concept. L'affichage de la carte et des territoire des civilisations a été testé et validé en jeu.
2. **Civilisations** : La simulation de plusieurs civilisations à été validé dans le prototype, Ainsi que leur évolution au court de la partie prenant en compte différentes conditions initiales.
3. **Nodes** : Grace à la simplification et l 'optimisation de tâches complexes au cœurs des *Nodes*, Le jeu peut actuellement en supporter une quantité acceptable simultanément. Le chargement du contenu au début de la partie à pu être accéléré en programmant la logique des *Nodes* directement en C# pour un fonctionnent plus efficace avec le reste du moteur.

Cependant, Chacun de ces aspects nécessite d'avantage d'attention afin de répondre au contraintes supplémentaires du produit fini : La logique de fonctionnement des civilisations et des *Nodes* a besoin d'être optimisé d'avantage pour accueillir le contenu supplémentaire jusqu'à la sortie du jeu. Et l'esthétique de la carte du monde nécessite d'être raffiné d'avantage afin de mitiger certains artefacts indésirables.

Tel sera notre objectif pour la version finale du jeu.

Outil Gaïa

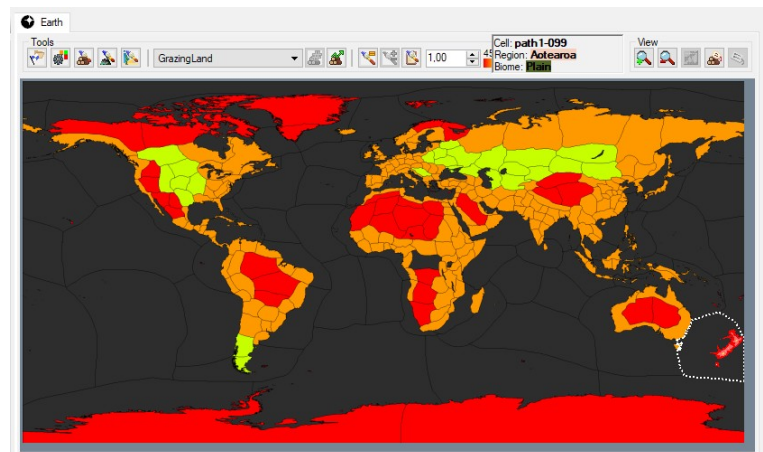


Gaïa est un logiciel spécialement créé pour le développement de Orbi Universo 2, Il répond au besoin d'édition des différents paramètres de la carte du monde.

- Importation de nouvelles régions
- Renommer les régions
- Configuration du climat.
- Répartition des ressources naturelles.
- Modification des caractéristiques géologiques.

Le logiciel permet d'ouvrir un fichier contenant les caractéristiques de la carte du monde utilisé par le jeu pour générer le terrain. Il suffit d'apporter des modifications dans Gaïa et de lancer une partie de Orbi Universo 2 pour immédiatement observer les changements. Rendement des itérations de la carte plus rapide.

Grace à un logiciel dédié comme Gaïa, l'édition et la modification de la carte est simplifiée et intuitive : les différentes propriétés peuvent être modifiées en « peignant » la carte, pour y ajouter les ressources naturelles par exemple. La teinte de la région change en fonction de la densité appliquée à chaque région.



L'affichage des données directement sur la carte permet d'obtenir une vue d'ensemble sur la répartition du climat et des ressources, facilitant les modifications.

A terme, L'interface de Gaïa devra être raffinée d'avantage pour permettre aux joueurs de l'utiliser et customiser la carte du monde à leur tour.